



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

SECR3253-02 NETWORK PROGRAMMING

Personal Reflection

Name	Matric No
KNAN FADI DAWARH	A23CS4013

Group Assignment – Network Automation Project

Group: Nahj

Personal Contribution

As part of this project, I was tasked with creating the Ansible playbook for collecting Linux system facts (`linux_facts.yml`). The purpose of this playbook was to automate the gathering of specific data points from a Linux system using Ansible, including the hostname, current date-and-time, CPU type(s), memory utilization and disk space.

The first task I had to complete was to educate myself on how Ansible interacts with Linux systems over SSH and how to collect facts using Ansible's built-in modules. I started reviewing different modules (`setup`, `command`, `shell`) to find the best fit for collecting the different attributes of system data.

Next, I organized the playbook in a manner that would allow all information to be integrated into a reporting template designed by someone else on the team.

While I was creating this playbook, I tested it several times to ensure it provided valid and consistent results. I also validated that the playbook executed properly on the Docker-based Ansible control node so that every member of the team could execute it without any variance in environments. Prior to merging with the main project playbook (`master site.yml`), I validated that my work adhered to the project's standards and would integrate seamlessly with the `master site.yml` playbook.

Besides implementing my assigned tasks, I actively participated in reviewing the project structure, discussing implementation approaches with my teammates, and helping troubleshoot issues related to Ansible execution and inventory configuration whenever they arose.

Challenges Encountered During Collaboration

Coordinating the development of various parts of the automation system developed by several members working on separate branches during the project was a major challenge. Because each member was responsible for different pieces of the automation system's source code, keeping the project organized and avoiding merge conflicts with all of the changes that would need to be merged into the master branch was critical.

Understanding how Ansible's various modules functioned posed additional challenges. Certain items could be acquired via the Ansible-supplied "facts." Other times, a Linux command had to be run to obtain pertinent data. Knowing when to utilize Ansible's built-in modules versus utilizing shell commands took a fair amount of trial and error and required reading official Ansible documentation.

Using Docker introduced an additional learning curve to Ansible and its operation. Running Ansible inside a container does not give you the same environment as running it directly on the host OS, so I had to learn how Ansible would manage its files, inventory, and SSH connections. Once I had learned to properly configure this environment, development was much more consistent between team members.

Another significant obstacle was performing testing. Many times, a small configuration error, such as installing an SSH key on the target server, would not allow the playbook to connect to and run on the target server. The process of troubleshooting these types of issues emphasized the need for the verification of each configuration step prior to assuming there was a problem in the playbook itself.

What I Learned

Participating in this project has enhanced my comprehension of network automation and infrastructure administration through Ansible greatly. Before this, I had few chances to apply Infrastructure as Code (IaC) in hands-on situations that would have provided practical exposure to the method of automating infrastructure via code. Building my playbook demonstrated that automation will take the place of many repetitive administrative duties and produce dependable and repeatable outcomes.

I learned how to create YAML playbooks, arrange Ansible projects, manage inventories, and acquire system data from remote Linux nodes as part of this experience. In addition, I discovered how using the Docker virtualization platform creates a standardized execution environment for program development and deployment. As a result, it allows developers on a project to collaborate more effectively since developers employ the same versions of the software used in the development effort and hence the same version of the software dependencies required to execute that code.

The project also enhanced my collaboration skills as an engineer through the collaborative use of Git and GitHub software engineering tool sets. Working off a distinct feature branch, checking in code changes regularly to the Git repository, and contributing to project via pull requests demonstrated how professional software development teams manage their code safely and efficiently. In addition, I gained a deeper insight of how the continuous integration processes functioned within the GitHub Actions workflow we implemented in our codebase after participating in developing the code for this project.

Overall, this project helped me appreciate the value of automation in modern IT environments. Tasks that would normally require manual configuration and system inspection can be completed consistently with a single playbook. Beyond the technical knowledge gained, the project improved my teamwork, communication, problem-solving, and version control skills, all of which will be valuable in future software engineering and network administration projects.